FASTAPI

FASTAPI FUNDAMENTALS

1.1:Course overview :

Welcome to the world of modern web development. The goal of this book is simple: to help you master the creation of high-performance REST APIs using Python's most powerful tool, FastAPI.

Before we dive in, ensure you have a basic grasp of Python (how to write simple scripts) and a general idea of REST (how data moves between a client and a server). With these basics in hand, you are ready to build professional-grade backends that are both fast to run and a joy to write.

1.2 Introducing FastAPI

1.2.1 FastAPI

In a very short time, FastAPI has "taken the Python world by storm." It has quickly become the gold standard for developers who need to build modern backends. But what makes it so special?

At its core, FastAPI is a high-performance framework designed specifically for building APIs. Its "magic" comes from two modern Python features:

- **Modern Python Integration:** It uses the latest, most efficient versions of Python.
- **Type Hints:** By simply telling the computer what kind of data to expect (like saying a name is "text"), FastAPI handles all the complex logic, security, and organization in the background automatically.

1.2.2 Key Features :

Fast API was built to solve two problems at once: making the computer work faster and making the developer more productive. Here is why it is the top choice for professionals today:

- **Blazing Speed:** It is one of the fastest Python frameworks available, performing on par with specialized languages like NodeJS and Go.
- **Built-in Safety:** Because it automatically validates your data, it reduces human errors by about **40%.** It catches mistakes before they become problems.
- **Rapid Development:** You can build features **200% to 300% faster** than with older tools. It does the "boring work" for you.
- **Effortless Coding:** Your code editor will offer smart suggestions and "autofill" features as you type, meaning you spend less time reading manuals and more time creating.
- **Ready for the Real World:** It produces "production-grade" code, meaning the apps you build here are strong enough to be used by real companies and thousands of users.
- **Globally Compatible:** It follows international standards (like OpenAPI and JSON Schema), ensuring your API works perfectly with every other modern tool on the internet.

1.2.3 Fast API vs. The other Python Web frameworks: Choosing the Right Tool

In the Python world, you will often hear about three main tools: **Django**, **Flask**, and **FastAPI**. To understand the difference, think of them as different ways to build a house.

- **Django** is like a fully furnished mansion—it comes with everything (a kitchen, a pool, security), but it can be heavy and slow if you only need a small room.
- **Flask** is like a basic toolkit; it gives you the hammer and nails, but you have to build every single wall yourself from scratch.
- **FastAPI** is like a modern, pre-fabricated studio—it is sleek, incredibly fast, and specifically designed for the modern world of data.

Feature	Django	Flask	FastAPI
Best For	Massive, complex websites.	Small, simple experiments.	**Modern, high-speed APIs.**

Feature	Django	Flask	FastAPI
Effort	High (lots of rules).	Medium (manual setup).	**Low (does the work for you).**
Outcome	Heavy and powerful.	Flexible but basic.	**Fast and professional.**

1.3 First Steps:

Now that we understand the power of FastAPI, it's time to get our hands dirty. Building your first project is a straightforward process that involves setting up a clean workspace and installing the necessary tools.

To start, you will follow these six simple steps:

1. **Choose Your Editor:** You can use any coding tool you like, such as **VS Code**, **PyCharm**, or **Cursor**.
2. **Create a Folder:** Start with a fresh folder on your computer to keep your project organized.
3. **Setup a Virtual Environment:** This creates a "private room" for your project so its settings don't interfere with other projects on your computer.
4. **Install FastAPI:** This is the core framework. We often add "extra dependencies" here to make sure we have all the professional features ready to go.
5. **Install Uvicorn:** Think of this as the "engine" that will actually run your FastAPI code so the world can see it.
6. **Create Your Endpoints:** This is where you write the actual Python code that handles data.

```python
from fastapi import FastAPI

app = FastAPI()
```

```python
@app.get("/date")
def date():
    """Return the current date/time."""
    return {'date': datetime.now()}
```

There no connection between the name of the function and url path name.

Understanding How FastAPI Works

There are a few "hidden secrets" to FastAPI that make it feel different from standard Python scripting:

➔ **Decorators and Naming**

- When you create a "bridge" (endpoint) for your data, you use something called a **Decorator**. One important thing for beginners to remember: **The name of your Python function does not have to match the URL.** You can name your function calculate_math() but set the URL path to /result. FastAPI gives you total freedom here.

➔ **The Automatic "Map": Swagger UI**

- One of the biggest reasons for FastAPI's massive popularity is the **Interactive Documentation**. By simply adding /docs to the end of your web address (for example: 127.0.0.1:8000/docs), FastAPI shows you a beautiful, interactive list of every endpoint you've created. You can test your API directly from this page without writing a single line of extra code.

➔ **Control Flow: The Request-Driven Model**

- In a normal Python script, the code runs from top to bottom and then stops. FastAPI is different. It doesn't execute your logic until an **Incoming HTTP Request** arrives (like when a user refreshes their browser).

- **The Engine:** Your code "sits" inside **Uvicorn**, which listens for requests.
- **The Trigger:** Every time a browser page is refreshed or a button is clicked, a new request is made, and only *then* does your specific function spring into action.

Why to use async: Going Faster

You will often see the words async def before a function in FastAPI. This is short for **Asynchronous**.

In older frameworks, the server would handle one task, finish it, and then move to the next. With async, FastAPI can start a task, and while it's waiting for data (like a response from a database), it can move on to help another user. This "multitasking" ability is what makes FastAPI extremely fast.

Note: If your application is just moving data around and isn't doing heavy mathematical calculations, always use async def. It allows your app to handle many more users at the same time.

1.4 Serving Data with FastApi:

In this section, we move beyond simple messages and learn how to serve real-world data. We will use a **List of Dictionaries** to represent a collection of cars, simulating how a real database would provide information to your app.

The Foundation: Our Data

Imagine we have a list of cars, each with a unique ID, a brand, and a size. In FastAPI, we can serve this entire list with a single "Read" operation.

Example:

```
from fastapi import FastAPI, HTTPException
from typing import Optional

app = FastAPI()

# Our "Fake Database"
CARS = [
    {"id": 1, "brand": "Tesla", "size": "medium", "doors": 4},
    {"id": 2, "brand": "Fiat", "size": "small", "doors": 2},
    {"id": 3, "brand": "Ford", "size": "large", "doors": 4},
]

@app.get("/cars")
async def read_all_cars():
    return CARS
```

Understanding Parameters: Query vs. Path

To make our API useful, we need to allow users to ask for *specific* data. There are two primary ways to do this in FastAPI: **Path Parameters** and **Query Parameters**.

1. Path Parameters: Finding a Specific Item

Path parameters are used when you want to identify a specific resource, like a single car by its ID. We define these inside the URL using curly braces {}.

- **How it works:** You put the name in the decorator (@app.get("/cars/{car_id}")) and then pass it as an argument to your function.
- **The 404 Error:** If a user asks for an ID that doesn't exist, we use HTTPException to politely tell them the data wasn't found.

- **Example:**

```python
@app.get("/cars/{car_id}")
async def get_car_by_id(car_id: int):
    # Find the car or raise an error
    for car in CARS:
        if car["id"] == car_id:
            return car
    raise HTTPException(status_code=404, detail="Car not found")
```

2. Query Parameters: Filtering Your Data

Query parameters are used for filtering or "searching" through your data. These are **not** defined in the URL path; instead, you simply list them as arguments in your function.

- **In the URL:** They appear after a question mark (e.g., /cars?size=small).
- **Optional Parameters:** By giving a parameter a default value like None, it becomes optional. The user doesn't *have* to provide it for the code to run.
- **Example:**

```python
@app.get("/cars/")
async def filter_cars(size: Optional[str] = None, doors: Optional[int] = None):
    results = CARS
    if size:
        results = [c for c in results if c["size"] == size]
    if doors:
        results = [c for c in results if c["doors"] == doors]
    return results
```

The Magic of Type Hints

You might have noticed code like car_id: int. These are **Type Hints**. While standard Python often ignores these, FastAPI uses them for three powerful "Magic" tricks:

1. **Validation:** If you say an ID must be an int (number) and a user types "abc", FastAPI will block the request and send an error automatically.
2. **Conversion:** If a user sends the number "5" as text in the URL, FastAPI automatically converts it into a real Python integer for you.
3. **Documentation:** These hints are what allow the /docs page to know exactly what kind of data your API expects.

Note: Type hints work for incoming data (input), but FastAPI does not use them to change what your function returns (output).

1.4 Serving Structured Data Using Pydantic Models:

As your application grows, using simple lists and dictionaries becomes messy. To build professional APIs, we need a "blueprint" for our data. In the world of FastAPI, we use **Pydantic Models** to design these blueprints (also called

Schemas)

What is a Pydantic Model?

Think of a Pydantic Model as a **digital mold**. If you want to create a "Car" object, the mold ensures every car has a brand, a price, and a year. If a piece of data doesn't fit the mold, FastAPI will reject it. This ensures your database stays clean and error-free.

```
from pydantic import BaseModel, Field
from typing import Optional

# This is our "Blueprint" or Schema
class Car(BaseModel):
    brand: str
    model: str
    year: int
    doors: int = 4  # This is a default value
```

The Four Pillars of HTTP Methods

To manage data, we use the four standard "actions" of the internet, known as HTTP methods:

Method	Action	Real-World Analogy
GET	Retrieve	Reading a page in a book.

Method	Action	Real-World Analogy
POST	Create	Writing a new page and adding it to the book.
PUT	Update	Erasing a sentence and writing a new one.
DELETE	Remove	Tearing a page out of the book.

Separation of Concerns: Input vs. Output

In a professional app, you don't always want the user to provide everything. For example, when **Creating** a car (POST), the user provides the details, but the **System** should generate the ID automatically.

By creating separate models for **Input** (what the user sends) and **Output** (what the user sees), you keep your data secure and automated.

```python
class CarInput(BaseModel):
    brand: str
    model: str
    year: int

class CarOutput(BaseModel):
    id: int  # The system generates this
    brand: str
    model: str
    year: int
```

Working with JSON Files

While we eventually use databases, using a **JSON file** is a great way to practice saving data.

- **Loading:** We use json.load() to read our cars from a file.
- **Saving:** Every time we add or update a car via a **POST** or **PUT** request, we rewrite the JSON file to keep our "database" current.

Note: In a real-world production app, we would use a proper database like PostgreSQL, but JSON files are perfect for learning the flow of data.

Delete: for deleting any item from database

Example: Post method to add a new car in database:

```python
@app.post("/api/cars/")
def add_car(car: Car):
    db.append(car)
    save_db(db)
```

And for adding the new item within the Json file created we will add a new function in schema file.

```python
def save_db(cars: list[Car]):
    with open("cars.json", 'w') as f:
        json.dump([car.model_dump() for car in cars], f, indent=4)
```

This is not a reliable and should not use it for production .

When we create a new item as the pydantic class fast Api API accept us to pass it as body of the class

Then when we add a new element it should be able to create a new id automatically that means we will not pass id with input instead we will get it as an output

```
class CarInput(BaseModel):
    fuel: str | None = "electric"
    doors: int
    transmission: str | None = "auto"


class CarOutput(CarInput):
    id: int


def load_db() -> list[CarOutput]:
    """Load a list of Car objects from a JSON file"""
    with open("cars.json") as f:
        return [CarOutput.model_validate(obj) for obj in json.load(f)]


def save_db(cars: list[CarOutput]):
    with open("cars.json", 'w') as f:
        json.dump([car.model_dump() for car in cars], f, indent=4)
```

```
@app.post("/api/cars/")
def add_car(car: CarInput) -> CarOutput:
    new_car = CarOutput(size=car.size, doors=car.doors,
                        fuel=car.fuel, transmission=car.transmission,
                        id=len(db)+1)
    db.append(new_car)
    save_db(db)
```

That define we can create separate input and output model that represents the same thing

Example: http delete method to delete the car by using id as path parameter:

```python
@app.delete(path: "/api/cars/{id}", status_code=204)
def remove_car(id: int) -> None:
    matches = [car for car in db if car.id == id]
    if matches:
        car = matches[0]
        db.remove(car)
        save_db(db)
    else:
        raise HTTPException(status_code=404, detail=f"No car with id={id}.")
```

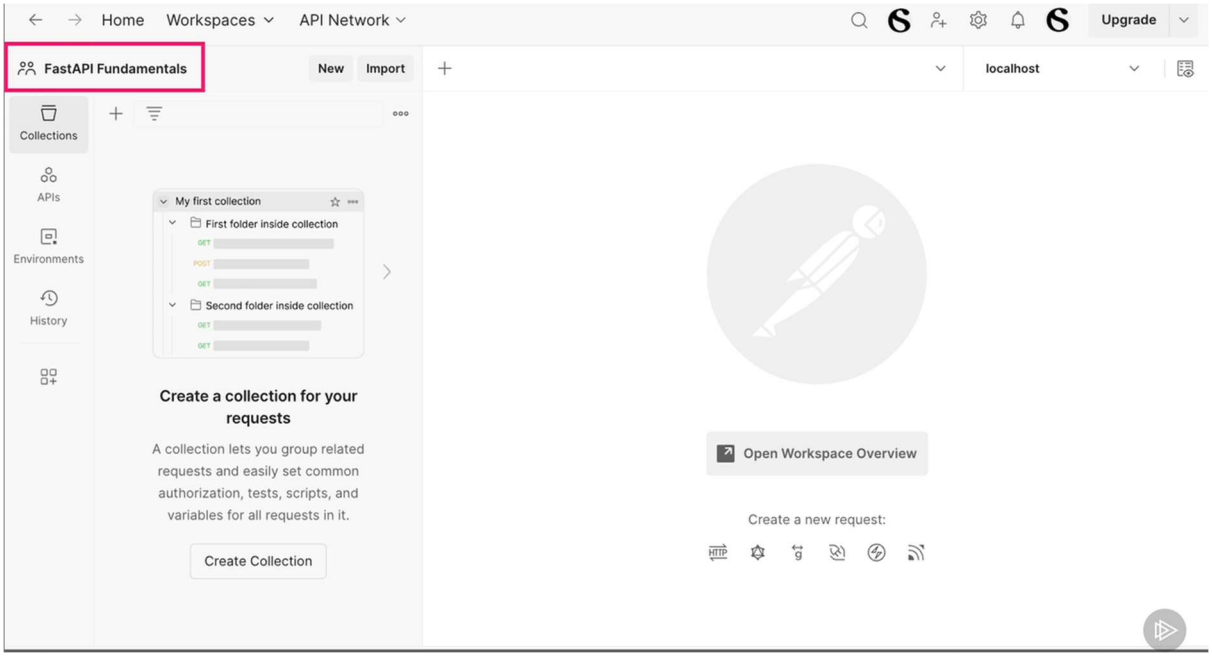
Example: Creating an http put method to update the value of the existing element

```python
@app.put("/api/cars/{id}")
def change_car(id: int, new_data: CarInput) -> CarOutput:
    matches = [car for car in db if car.id == id]
    if matches:
        car = matches[0]
        car.fuel = new_data.fuel
        car.transmission = new_data.transmission
        car.size = new_data.size
        car.doors = new_data.doors
        save_db(db)
        return car
    else:
        raise HTTPException(status_code=404, detail=f"No car with id={id}.")
```

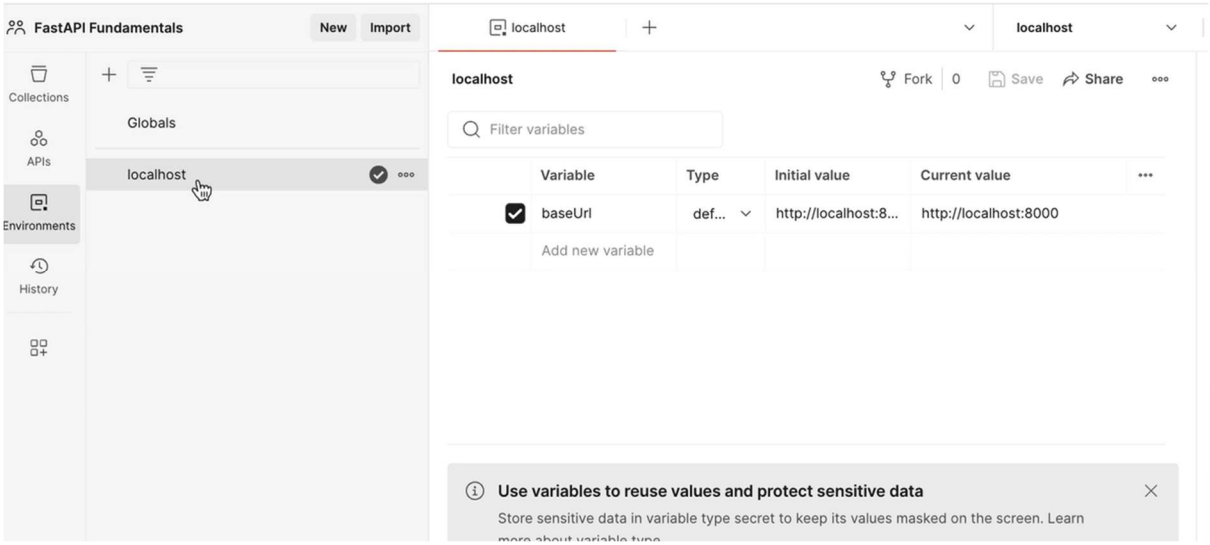
Testing with Postman

Writing the code is only half the battle; you also need to test it. **Postman** is a tool that acts like a browser but is designed specifically for developers.

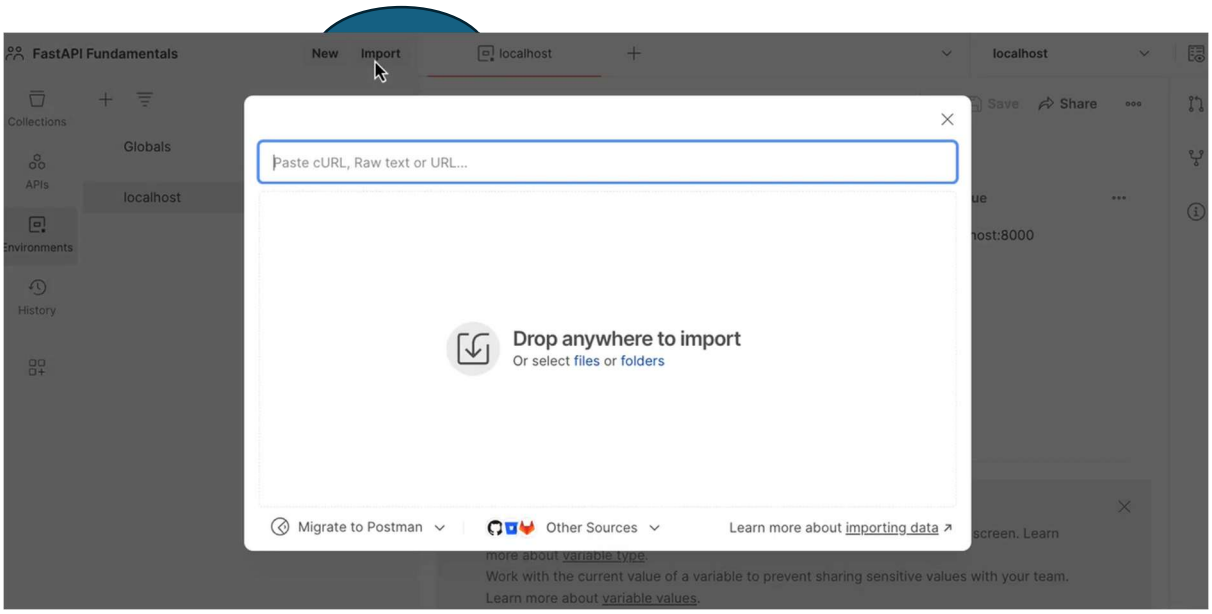
1. **Setup:** Create a workspace and set your "Base URL" (usually http://127.0.0.1:8000).
2. **Request Body:** When using **POST** or **PUT**, you will send your data as a "JSON Body."
3. **Automation:** You can save your requests so you don't have to type the data over and over again.



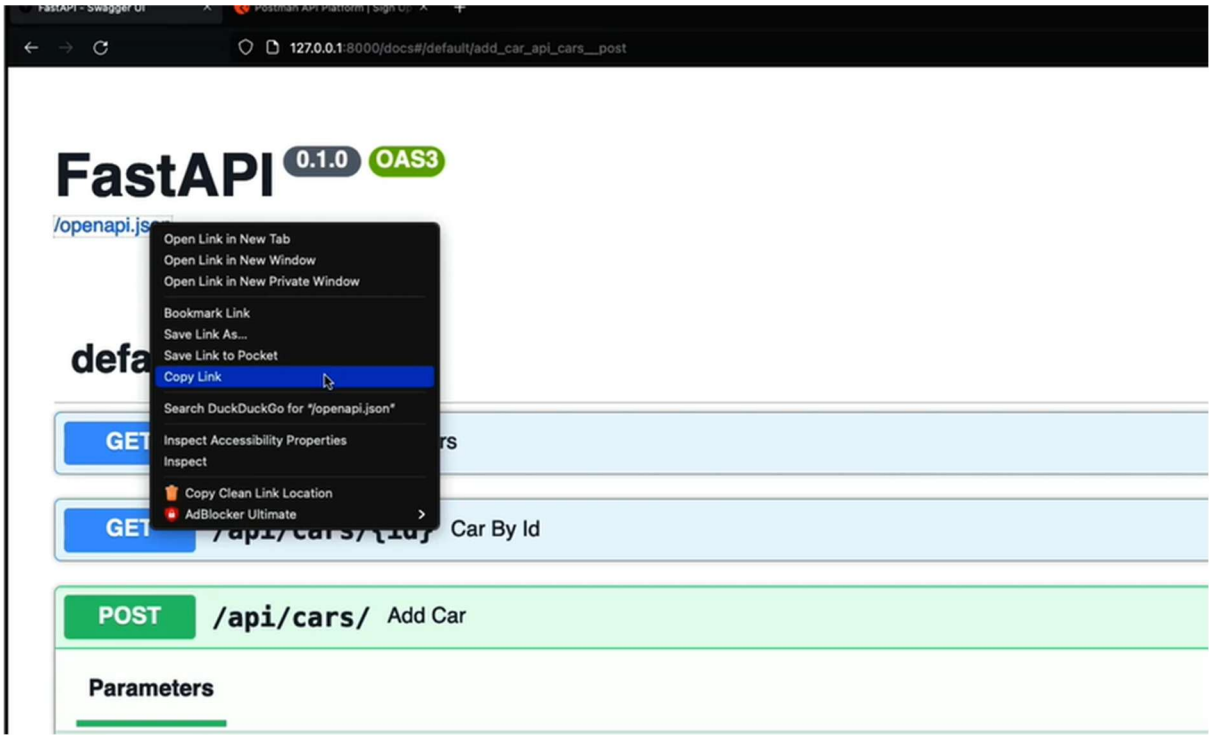
1. Configuring postman: setting up the environments , where you can give the base url of the local used



2. As we will be using JSON file then attach you JSON file with postman so that it can used while execution



3. Import the JSON file by copy the link from browser localhost.

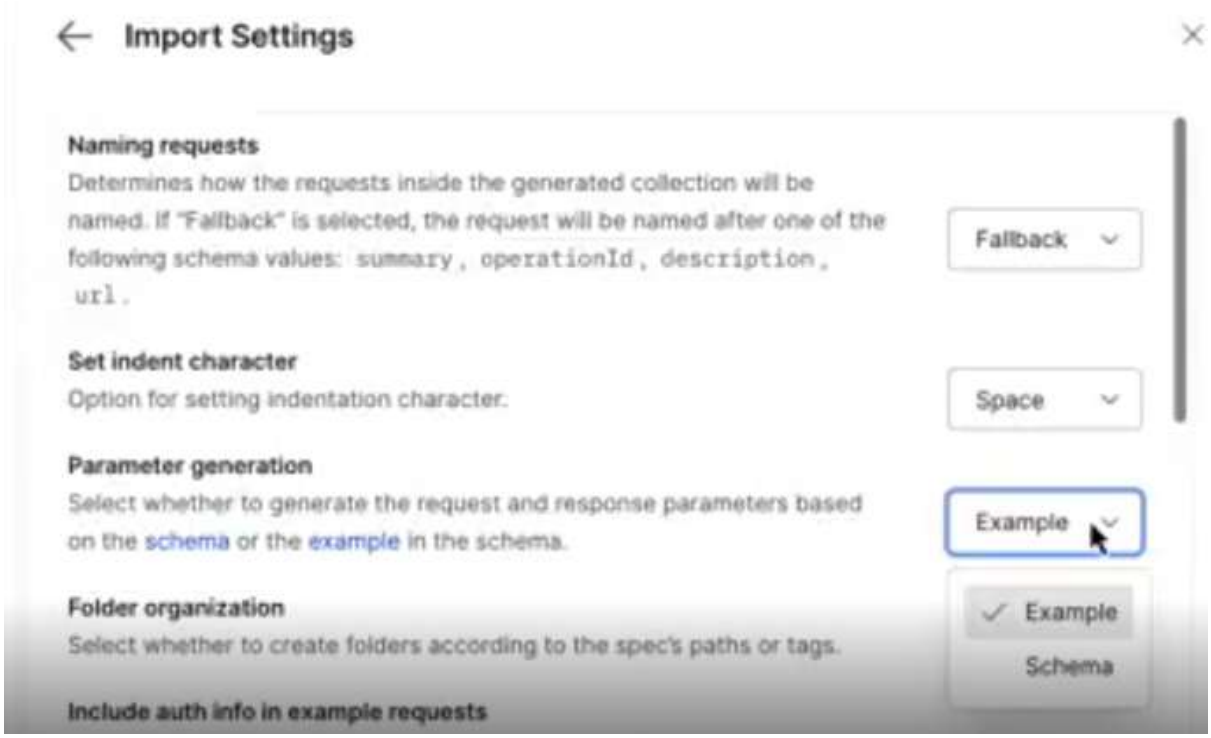


4. Then can use various http methods by giving appropriate attributes

Adding data every time while making the request is sometimes annoying that is why instead we can add some example data by creating model_config. It helps giving value to the program by giving some examples

```
model_config = {
    "json_schema_extra": {
        "examples": [
            {
            "size": "m",
            "doors": 5,
            "transmission": "manual",
            "fuel": "hybrid"
```

After adding the model config we need import JSON again but this time with some extra settings



Adding Example Data:

To make your documentation (/docs) even more helpful, you can add a model_config to your Pydantic class. This provides "Example Data" that appears automatically in Swagger and Postman, so other developers know exactly what a "Car" should look like

```python
class Car(BaseModel):
    brand: str
    model: str

    model_config = {
        "json_schema_extra": {
            "example": {
                "brand": "Tesla",
                "model": "Model 3"
            }
        }
    }
```

1.6 Using a Database with fastAPI

In previous chapters, our application stored data in memory. This meant that every time the server restarted, all data was lost.

Two Worlds of Data

When working with databases, data exists in two different worlds:

1. **Python World**
    - Data is represented using classes and objects
2. **Database World**
    - Data is stored in tables, rows, and columns

Bridging the Gap with SQLModel

To connect these two worlds, we use **SQLModel**.

SQLModel is a powerful library created by Sebastián Ramírez that combines:

- **SQLAlchemy** → Database ORM (Object Relational Mapping)

- **Pydantic** → Data validation and serialization

🧩 Key Insight

SQLModel allows us to:

- Define database models and validation schemas in one place
- Write cleaner and less repetitive code

Note: SQLModel primarily uses synchronous operations. Even so, FastAPI can still handle requests concurrently.

🧩 Setting Up the Foundation

Database-driven applications require some initial setup (often called boilerplate).

Using Lifespan for Initialization

FastAPI provides a **lifespan handler** to run logic during:

- Application startup
- Application shutdown

Purpose of Lifespan

Before the application starts handling requests:

- Ensure database connection is ready
- Create required tables

How It Works

- Define a lifespan function using @asynccontextmanager
- Call:
  - SQLModel.metadata.create_all(engine)
- Attach it to FastAPI:
  - FastAPI(lifespan=lifespan)

Execution Flow

- App starts
- Lifespan function runs

- Tables are created
- App begins serving requests

It was created by the same creator (sebastin ramirez)

SQLModel:

This is a layer over sqlalchemy and it use use the underlying the layer to do the query

Note: fastapi does not use async functions with sql model , and we will be using regular functions and still fastapi is going to work concurrently.

Creating Database Models

Steps:

Step 1: Install SQLModel

Install the required package.

```
python -m pip install sqlmodel
```

Step 2: Define Models

Instead of plain Pydantic models, we use SQLModel classes.

```
from sqlmodel import SQLModel, Field
    id: int


class CarInput(SQLModel):
    size: str
    fuel: str | None = "electric"
    doors: int
    transmission: str | None = "auto"

    model_config = {
        "json_schema_extra": {
            "examples": [
                {
```

Key Concept

- table=True → Marks the class as a database table
- Field() → Defines column properties

```
class Car(CarInput, table=True):
    id: int | None = Field(primary_key=True, default=None)
```

Setting Up the Database Engine

To connect to the database:

- Import create_engine from SQLAlchemy
- Create an engine instance

This engine acts as the **entry point to the database**.

```
from sqlalchemy import create_engine

engine = create_engine(
    "sqlite:///carsharing.db",
    connect_args={"check_same_thread": False},  # Needed for SQLite
    echo=True  # Log generated SQL
)
```

This is the boiler plate code that we usually copy and paste

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    SQLModel.metadata.create_all(engine)
    yield
```

This is the lifespan handler function for a fastapi application to manage application startup and shutdown events, specifically for database initialization , where:

- **@asynccontextmanager**: Decorates the function to treat it as an asynchronous context manager.

- **async def lifespan(...)**: Defines the asynchronous function to run on startup and shutdown.
- **SQLModel.metadata.create_all(engine)**: Executes database table creation based on defined models using the provided engine.
- **yield**: Pauses execution to allow the application to run, resuming afterward to handle shutdown procedures.

```
app = FastAPI(title="Car Sharing", lifespan=lifespan)
```

It is to be attached with program by passing lifespan=lifespan

The effect of this : the fastapi will call the lifespan function to run anything that needs to be run before we will start handling request

Creating Records (POST Operation)

Flow

1. Receive request
2. Validate input
3. Convert input to database model
4. Save to database
5. Return stored object

```
@app.post("/api/cars/")
def add_car(car_input: CarInput) -> Car:
    with Session(engine) as session:
        new_car = Car.model_validate(car_input)
        session.add(new_car)
        session.commit()
        session.refresh(new_car)
        return new_car
```

Code breakdown :

1. **@app.post("/api/cars/")**: This is a FastAPI decorator. It tells the app to trigger the function below whenever a POST request is sent to the /api/cars/ URL.
2. **def add_car(car_input: CarInput) -> Car:**:
    1. car_input: CarInput: FastAPI automatically validates the incoming JSON body against the CarInput schema.
    2. -> Car: This hints that the function will return an object of type Car.
3. **with Session(engine) as session:**: This creates a database session that automatically closes when the code block finishes (even if an error occurs).
4. **new_car = Car.model_validate(car_input)**: This converts the input data (likely a Pydantic model) into a database model instance (Car).
5. **session.add(new_car)**: Places the new car object into the session's "to-be-saved" list.
6. **session.commit()**: Flushes the changes to the actual database.
7. **session.refresh(new_car)**: Re-reads the car data from the database. This is important to get the database-generated id.
8. **return new_car**: Sends the saved car data back to the user as a JSON response.

The idea of database transaction is group bunch of car together , if any exception happens nothing will be written and no error everything is to be written.

Key Steps Explained

- **Input Validation**
    - FastAPI validates request body using schema
- **Session Management**
    - Use Session(engine) to interact with DB
- **Insert Data**
    - session.add()
- **Commit Transaction**
    - session.commit()

- **Refresh Object**
    - session.refresh() → fetch generated values (like ID)

Transaction Concept

A **transaction** ensures:

- All operations succeed → changes saved
- Any failure → nothing is saved

This guarantees data consistency.

How to implement http operation by using sql:

a. get:

```
@app.get("/api/cars")
def get_cars(size: str | None = None, doors: int | None = None) -> list:
    with Session(engine) as session:
        query = select(Car)
        if size:
            query = query.where(Car.size == size)
        if doors:
            query = query.where(Car.doors >= doors)
        return session.exec(query).all()
```

b. Post:

```
@app.post("/api/cars/")
def add_car(car_input: CarInput) -> Car:
    with Session(engine) as session:
        new_car = Car.model_validate(car_input)
        session.add(new_car)
        session.commit()
        session.refresh(new_car)
        return new_car
```

c. Delete method:

```python
@app.delete( path: "/api/cars/{id}", status_code=204)
def remove_car(session: Annotated[Session, Depends(get_session)],
               id: int) -> None:
    car = session.get(Car, id)
    if car:
        session.delete(car)
        session.commit()
    else:
        raise HTTPException(status_code=404, detail=f"No car with id={id}.")
```

d. Put method:

```python
@app.put("/api/cars/{id}")
def change_car(session: Annotated[Session, Depends(get_session)],
               id: int, new_data: CarInput) -> Car:
    car = session.get(Car, id)
    if car:
        car.fuel = new_data.fuel
        car.transmission = new_data.transmission
        car.size = new_data.size
        car.doors = new_data.doors
        session.commit()
        return car
    else:
        raise HTTPException(status_code=404, detail=f"No car with id={id}.")
```

Reducing Repetition with Dependency Injection

Problem

We repeatedly write:

- with Session(engine) as session

```python
def get_session():
    return Session(engine)


app = FastAPI(title="Car Sharing", lifespan=lifespan)
db = load_db()

@app.get("/api/cars")
def get_cars(session: Annotated[Session, Depends(get_session)],
             size: str | None = None, doors: int | None = None) -> list:
    with Session(engine) as session:
```

Solution: Dependency Injection

Create a reusable function:

- Returns a database session
- Automatically manages lifecycle

How It Helps

- Cleaner code
- Avoid repetition
- Centralized session handling

Key Components

- Depends (from FastAPI)
- Annotated (for type hints)

Extending the Model: Adding Trips

To expand functionality, we introduce a **Trip** table.

```python
class Trip(TripInput, table=True):
    id: int | None = Field(default=None, primary_key=True)
    car_id: int = Field(foreign_key="car.id")
    car: "Car" = Relationship(back_populates="trips")
```

Why Add Trip?

- Represents relationships between entities
- Enables more complex queries

Relationship Concept

- A **Trip** is linked to other entities (e.g., Car)
- Uses back_populates to establish bidirectional relationships

What is back_populates?

It connects two models so that:

- Each knows about the other
- Enables navigation between related data

Creating Trip Endpoints

- Define new POST endpoint
- Accept trip input
- Store in database
- Return response

```python
@app.post("/api/cars/{car_id}/trips")
def add_trip(session: Annotated[Session, Depends(get_session)],
             car_id: int, trip_input: TripInput) -> Trip:
    car = session.get(Car, car_id)
    if car:
        new_trip = Trip.model_validate(trip_input, update={'car_id': car_id})
        car.trips.append(new_trip)
        session.commit()
        session.refresh(new_trip)
        return new_trip
    else:
        raise HTTPException(status_code=404, detail=f"No car with id={id}.")
```

1.7 Working with HTTP and fastapi

FastAPI provides comprehensive support for handling HTTP requests, enabling developers to build both **API services** and **web-based interfaces**.Topics

In this section, we explore:

- Custom response types
- Serving HTML content
- Dynamic templates with Jinja2
- Form data processing
- Status codes and error handling
- API structuring with routers
- Middleware concepts (including CORS)

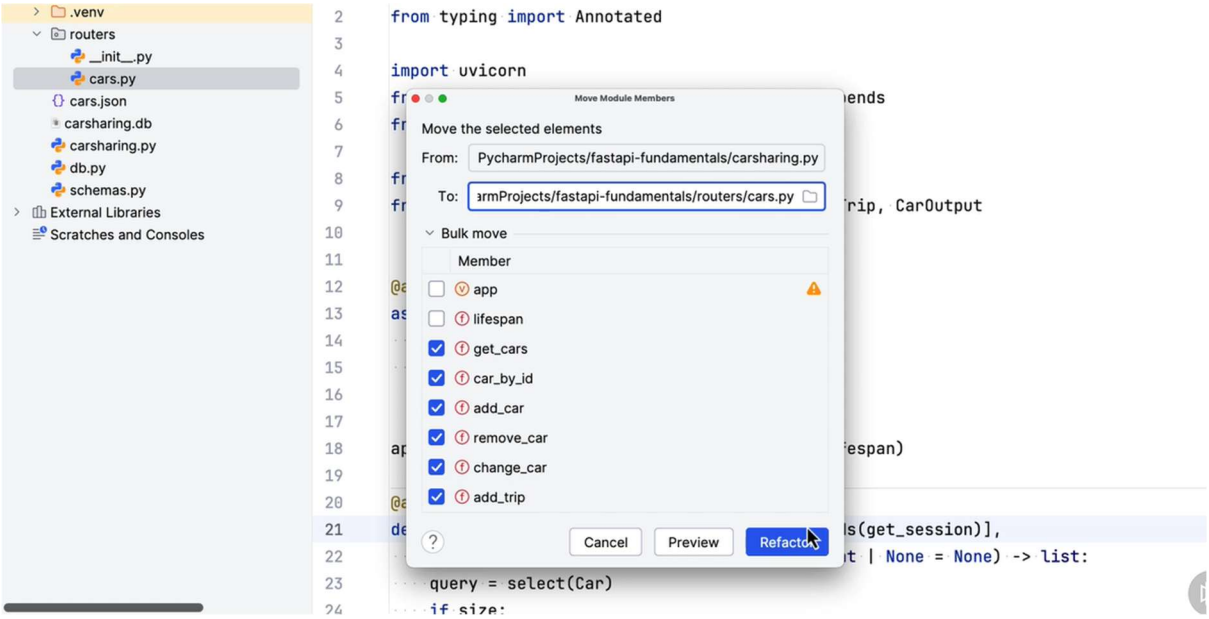
🧩 Structuring APIs with APIRouter

As applications grow, placing all endpoints in a single file becomes difficult to manage.

Refactoring into Routers

- Create a new package:
  - routers/
- Move all endpoint functions into router files
- Replace:
  - app → router

- Move all endpoint functions into router files



  - Use **APIRouter** instead of FastAPI instance

```
from fastapi import Depends, HTTPException, APIRouter

router = APIRouter()
```

  - **Registering Routers in Main Application**

In main.py:

- Import routers
- Register them using:
  - app.include_router(router)

```
app = FastAPI(title="Car Sharing", lifespan=lifespan)
app.include_router(cars.router)
```

  - **Benefits**
    - Modular design
    - Improved readability

- Easier scaling

- **Adding Common Prefix**

Routers allow grouping endpoints:

- Example: /cars, /products

This ensures consistent API structure.

```
router = APIRouter(prefix="/api/cars")
```

Serving HTML Content

FastAPI is not limited to APIs—it can also serve web pages.

Basic Approach

You can return HTML directly from endpoints.

However:

Writing HTML inside Python strings is not scalable or maintainable.

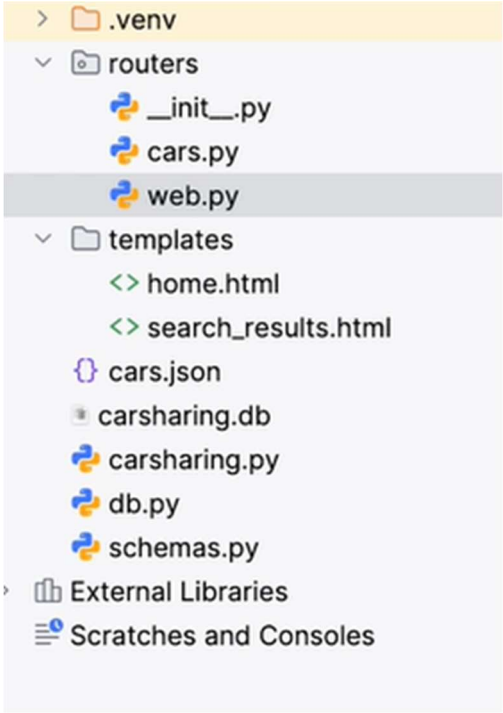
```python
router = APIRouter()

@router.get(path: "/", response_class=HTMLResponse)
def home():
    return """
    <html>
        <head>
            <title>Carsharing Demo</title>
        </head>
        <body>
            <h1>Welcome to the Car Sharing service</h1>
            <p>Here is some text for you</p>
        </body>
    </html>
    """
```

Dynamic HTML with Jinja2

To properly manage HTML, we use **Jinja2 templating engine**.

Setup

1. Create a templates/ folder
2. Store HTML files separately



Connecting Templates with Endpoints

- Use endpoints (usually GET/POST)
- Pass data from backend to template

carsharing.py db.py cars.py web.py schemas.py

```python
from fastapi import APIRouter, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates

router = APIRouter()

templates = Jinja2Templates(directory="templates")


@router.get( path: "/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse("home.html",
                                      {"request": request})

```

Example Use Case: Search Page

- Accept input from user
- Query database
- Return results rendered in HTML

```
@router.post( path: "/search", response_class=HTMLResponse)
def search(size: Annotated[str, Form()],
           doors: Annotated[int, Form()],
           request: Request,
           session: Annotated[Session, Depends(get_session)]):
    cars = get_cars(size=size, doors=doors, session=session)
    return templates.TemplateResponse("search_results.html",
                                      {"request": request, "cars": cars})
```

Code breakdown :

- **Endpoint Path:** The function handles requests to the /search path.
- **Functionality:** It accepts form data for size (as a string) and doors (as an integer), then fetches car data based on these criteria.
- **Response:** It returns an HTML page titled search_results.html populated with the search results.

Handling Form Data

Form data is different from JSON.

To process it correctly:

- Use FastAPI form handling utilities
- Explicitly declare form fields

📌 Jinja2 Template Rendering

Jinja allows dynamic content generation.

Example Loop

- {% for car in cars %} → Iterates over list
- {{ car }} → Injects value

- {% endfor %} → Ends loop

```html
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Carsharing: Search Results</title>
</head>
<body>
<p>We found the following matching cars:</p>
<ul>
    {% for car in cars %}
    <li>
        {{car}}
    </li>
    {% endfor %}
</ul>
</body>
</html>
```

- **{% for car in cars %}**: This is a "for loop." It tells the server to look at a list called cars and repeat the code inside the loop for every single car in that list.
- **<li> {{car}} </li>**:
    - <li> creates a list item.
    - {{car}} is a placeholder. It will be replaced by the actual name/data of the car (e.g., "Tesla Model 3", "Toyota Prius").
- **{% endfor %}**: This tells the engine where the loop ends.

Why Use Jinja2?

- Clean separation of frontend and backend
- Dynamic page generation
- Reusable templates

Status Codes and Error Handling

Default Behavior

FastAPI automatically returns status codes, but:

- They may not always be meaningful

Custom Status Codes

You can explicitly:

- Return status codes using response objects
- Raise errors using HTTPException

```
@app.exception_handler(BadTripException)
async def unicorn_exception_handler(request: Request, exc: BadTripException):
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content={"message": "Bad Trip"},
    )
```

Why This Matters

- Improves API clarity
- Helps frontend handle responses correctly
- Aligns with REST standards

Middleware in FastAPI

What is Middleware?

Middleware is a function that:

- Runs before request processing
- Runs after response generation

Use Cases

- Logging
- Authentication
- Request modification
- Response transformation

Example:

```python
import time

from fastapi import FastAPI, Request

app = FastAPI()


@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

There are many types of middleware

Middleware function for our applications

```python
@app.middleware("http")
async def add_cars_cookie(request: Request, call_next):
    response = await call_next(request)
    response.set_cookie(key="cars_cookie", value="you_visited_the_carsharing_app")
    return response
```

Example: Cookies

Middleware can:

- Set cookies
- Modify headers
- Track sessions

```python
@router.get( path: "/", response_class=HTMLResponse)
def home(request: Request,
         cars_cookie: Annotated[str | None, Cookie()]):
    print(cars_cookie)
    return templates.TemplateResponse("home.html",
                                      {"request": request})
```

CORS Middleware (Cross-Origin Resource Sharing)

Problem Scenario

You create:

- A frontend HTML page (running on one port, e.g., 8080)
- A FastAPI backend (running on another port, e.g., 8000)

What Happens?

Browser blocks the request due to:

CORS policy: Missing Allow-Origin

Why This Happens

Browsers enforce **same-origin policy**:

- Prevents unauthorized cross-origin requests

Solution: Enable CORS Middleware

- Add CORS middleware in FastAPI
- Configure allowed origins

Steps to check :

```
⇒ python -m http.server 8080
```

This command will run the http server and will run any file found in the project directory.

When you will open localhost:8080

Directory listing for /

- .DS_Store
- .git/
- .idea/
- __pycache__/
- cars.json
- carsharing.db
- carsharing.py
- cors_demo/
- db.py
- routers/
- schemas.py
- templates/

And open cors_demo.html page: this page is going the page with the instruction just executed and also gives a button to open local:8000 where api calls were done

Use the following command to serve this file:

```
python -m http.server 8080
```

This will server the current file at http://localhost:8080.

The button below will send a request to the API at http://localhost:8000/api/cars.

Klik me!

But this is not going to work and display failed and in network it is going to return CORS missing allow origin this happens because of security and only renders the file from the same origin

In carsharing main page we will declare middleware

```
27    ]
28
29    app.add_middleware(
30        CORSMiddleware,
31        allow_origins=origins,
32        allow_credentials=True,
33        allow_methods=["*"],
34        allow_headers=["*"],
35    )
36
```

Demonstration: CORS Behavior

Steps

1. Create cors_demo.html
2. Run a local server:
    - Example: port 8080
3. Open the HTML page
4. Trigger API request to FastAPI server

Observation

- Request fails initially
- Error: CORS missing allow origin

After Enabling CORS

- Request succeeds
- Data is returned correctly

Serving Static and HTML Content

FastAPI supports:

- Static files (CSS, JS)
- HTML templates

This allows building lightweight full-stack applications.

1.8 Adding Authentication

In real-world applications, not every user should have unrestricted access. We need a way to:

- Identify users (**authentication**)
- Protect sensitive operations
- Secure stored credentials

This section introduces authentication in FastAPI, covering:

- User modeling
- Password hashing
- HTTP Basic Authentication
- OAuth2-based authentication

Creating the User Model

To support authentication, we first define a **User** entity.

```
class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str
    password_hash: str = ""
```

```
class UserOutput(SQLModel):
    id: int
    username: str
```

Database Model (SQLModel)

A typical user model includes:

- **id** → Primary key
- **username** → Unique identifier
- **password_hash** → Securely stored password

Key Features

- table=True → Marks it as a database table
- Indexed fields → Improve query performance
- Unique constraint → Prevent duplicate usernames

Why Not Store Plain Passwords?

Storing raw passwords is extremely dangerous.

Best Practice: Always store hashed passwords, never plain text.

- **Password Hashing with bcrypt**

What is Hashing?

Hashing converts a password into a secure, irreversible format.

Why bcrypt?

- Industry-standard for password hashing
- Built-in salting
- Resistant to brute-force attacks

Core Functions

- **Hash Password**
  - Convert plain password → hashed password
- **Verify Password**
  - Compare input password with stored hash

Step-by-step implementation:

- Installing bcrypt module

```
> python -m pip install "passlib[bcrypt]"
```

- And adding two function to the user class that take password and get the hashed password and store the password in hashed format

```python
from sqlmodel import SQLModel, Field, Relationship

from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"])

class UserOutput(SQLModel):
    id: int
    username: str


class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str
    password_hash: str = ""

    def set_password(self, password):
        """Setting the passwords actually sets password_hash."""
        self.password_hash = pwd_context.hash(password)

    def verify_password(self, password):
        """Verify given password by hashing and comparing to password_hash."""
        return pwd_context.verify(password, self.password_hash)
```

Verify function check the hash password is safe

Adding functionality to not have duplicate username

```python
class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str = Field(sa_column=Column(__name_pos: "username", VARCHAR, unique=True, index=True))
    password_hash: str = ""
```

Workflow

1. User registers
2. Password is hashed

3. Hashed value is stored in database

During login:

1. User enters password
2. System verifies against stored hash

📌 Preventing Duplicate Users

To ensure uniqueness:

- Add constraint on username
- Validate before insertion

Why This Matters

- Avoid identity conflicts
- Maintain data integrity

📌 HTTP Basic Authentication

What is Basic Auth?

A simple authentication method where:

- Browser prompts user for credentials
- Credentials are sent with each request

How It Works

1. Client accesses protected endpoint

```python
from typing import Annotated

from fastapi import Depends, HTTPException
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from sqlmodel import Session, select
from starlette import status

from db import get_session
from schemas import UserOutput, User

security = HTTPBasic()


def get_current_user(
        credentials: Annotated[HTTPBasicCredentials, Depends(security)],
        session: Annotated[Session, Depends(get_session)]
) -> UserOutput:
    query = select(User).where(User.username == credentials.username)
    user = session.exec(query).first()
    if user and user.verify_password(credentials.password):
        return UserOutput.model_validate(user)
    else:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
```

2. Browser shows login popup

```python
@router.post("/")
def add_car(session: Annotated[Session, Depends(get_session)],
            user: Annotated[User, Depends(get_current_user)],
            car_input: CarInput) -> Car:
    new_car = Car.model_validate(car_input)
    session.add(new_car)
    session.commit()
    session.refresh(new_car)
    return new_car
```

- User enters username/password
- Server verifies credentials

Limitation

Basic Auth is **not secure** for production:

- Credentials are sent repeatedly
- Requires HTTPS to be safe

🧩 Nested Dependencies in FastAPI

FastAPI allows chaining dependencies.

Concept

- One dependency can depend on another
- Example:
    - Get user → verify credentials → allow access

Why This is Powerful

- Reusable authentication logic
- Clean separation of concerns

OAuth2 Authentication

To build secure systems, we use **OAuth2-based authentication**.

Why OAuth2?

- Token-based authentication
- Stateless and scalable
- Industry standard

How It Works

1. User logs in
2. Server generates a token

3. Client stores token
4. Token is sent with future requests

```python
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from sqlmodel import Session, select

URL_PREFIX="/auth"
router = APIRouter(prefix=URL_PREFIX)
oauth2_scheme = OAuth2PasswordBearer(tokenUrl=f"{URL_PREFIX}/token")
```

```python
def get_current_user(token: Annotated[str, Depends(oauth2_scheme)],
                     session: Annotated[Session, Depends(get_session)]) -> UserOutput:
    query = select(User).where(User.username == token)
    user = session.exec(query).first()
    if user:
        return UserOutput.model_validate(user)
    else:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Username or password incorrect",
            headers={"WWW-Authenticate": "Bearer"},
        )
```

Advantages Over Basic Auth

- No repeated credential transmission
- Supports expiration
- More secure and flexible

🔐 Authentication Flow in FastAPI

Login Process

1. User sends credentials
2. Server validates user
3. Token is generated
4. Token returned to client

Protected Endpoints

- Require valid token
- Use dependency injection to validate token

Authentication Enforcement

Example scenario:

Before creating a resource (e.g., a car), the user must be authenticated.

```
@router.post("/token")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
                session: Annotated[Session, Depends(get_session)]):
    query = select(User).where(User.username == form_data.username)
    user = session.exec(query).first()
    if user and user.verify_password(form_data.password):
        return {"access_token": user.username, "token_type": "bearer"}
    else:
        raise HTTPException(status_code=400, detail="Incorrect username or password")
```

Implementation Idea

- Protect endpoint using dependency
- Validate user before execution

1.9 Testing and Deploying

Building an API is only part of the job. A production-ready system must also be:

- **Tested** → to ensure correctness
- **Reliable** → behaves as expected under different conditions
- **Deployable** → accessible in real-world environments

This section covers:

- Unit testing using pytest
- Testing APIs with FastAPI TestClient
- Mocking dependencies
- Deploying using Linux and Nginx

🧪 Unit Testing with pytest

What is Unit Testing?

Unit testing verifies that individual parts of your application:

- Work correctly
- Follow expected behavior

Working:

- Step 1: Install pytest
    - Install the testing framework
    - Used to write and execute test cases

```
python -m pip install pytest
```

- Step 2: Create Test Directory
    - Create a tests/ folder
    - Store all test files here

Naming Convention

- Test files → test_*.py
- Test functions → test_*

🧪 Testing APIs with TestClient

FastAPI provides a built-in testing utility.

How It Works

- Simulates HTTP requests
- No need to run a live server

Basic Example: Testing Homepage

Test Flow:

1. **Import TestClient**
2. **Import FastAPI app**

3. **Create client instance**
4. **Send request**
5. **Validate response**

carsharing.py test_home.py

```python
from fastapi.testclient import TestClient

from carsharing import app

client = TestClient(app)


def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert "Welcome" in response.text
```

What This Test Verifies

- Endpoint is reachable
- Status code is correct (200 OK)
- Response content is valid

🧪 Testing API Responses

Schema Validation

When testing endpoints like /api/cars/:

- Ensure response is a list
- Ensure each item contains required fields

Example Checks

- Each object must include:
    - doors
    - size

```python
def test_get_cars():
    response = client.get("/api/cars/")
    assert response.status_code == 200
    cars = response.json()
    assert all(["doors" in c for c in cars])
    assert all(["size" in c for c in cars])
```

Code breakdown:

- **The API Structure**: The /api/cars/ endpoint is expected to return a list of dictionaries (JSON objects).
- **Schema Validation**: The test is ensuring that the data follows a specific "schema" or contract. Even if the values are different, every car must at least have these two specific fields: doors and size.
- **Pass/Fail Logic**:
    - If **all** cars have these keys, the test continues.
    - If **even one** car is missing either "doors" or "size", the all() function will return False, and the assert statement will raise an AssertionError, causing the test to fail.

Why This Matters

- Enforces API contract
- Prevents breaking frontend integrations

Testing POST Requests

Simulating Requests

- Use client.post()
- Pass JSON body
- Include headers if needed

```python
from unittest.mock import MagicMock
from fastapi.testclient import TestClient

from carsharing import app
from routers.cars import add_car
from schemas import CarInput, User, Car

client = TestClient(app)


def test_add_car():
    response = client.post(url: "/api/cars/",
                           json={
                               "doors": 7,
                               "size": "xxl"
                           }, headers={'Authorization': 'Bearer reindert'}
                           )
    assert response.status_code == 200
    car = response.json()
    assert car['doors'] == 7
    assert car['size'] == 'xxl'
```

Code breakdown:

- **client.post(...)**: This sends a simulated **POST** request to the application.
- **url="/api/cars/"**: This is the endpoint (route) being tested.
- **json={...}**: This provides the request body. The TestClient automatically serializes this dictionary into a JSON string and sets the Content-Type header to application/json.
- **headers={"Authorization": "Bearer reindert"}**: This simulates a request from an authenticated user. It sends a **JWT (JSON Web Token)** or similar bearer token to the server to pass through security dependencies.

Authentication Testing

- Add header:
    - Authorization: Bearer <token>

Purpose

- Validate request handling
- Verify authentication flow

- Ensure correct response

🔌 Mocking Dependencies

Why Mocking?

In unit tests:

- We should not depend on real databases

Solution: Mock Objects

- Replace real database session with a mock
- Simulate behavior

Test Structure (AAA Pattern)

1. Arrange

- Create mock session
- Prepare input data

2. Act

- Call function under test

3. Assert

- Verify behavior

Example:

```
def test_add_car_with_mock_session():
    mock_session = Mock()
    input = CarInput(doors=2, size="xl")
    user = User(username="reindert")
    result = add_car(car_input=input, session=mock_session, user=user)

    mock_session.add.assert_called_once()
    mock_session.commit.assert_called_once()
    mock_session.refresh.assert_called_once()
    assert isinstance(result, Car)
    assert result.doors == 2
    assert result.size == "xl"
```

Code breakdown:

1. Setup (Arrange)

- **mock_session = Mock()**: This creates a mock object that acts as a stand-in for a real database session (like a SQLAlchemy session). Since it's a Mock, it will record every time a method is called on it.
- **input & user**: These lines initialize the data objects (DTOs or Models) that will be passed into the function being tested.

2. Execution (Act)

- **result = add_car(...)**: The function add_car is called. It is passed the mock_session. Inside add_car, the code likely performs some logic and then calls methods like .add(), .commit(), and .refresh() on that session.

3. Verification (Assert)

The test uses assertions to ensure the function behaved as expected:

- **Call Assertions**:
  - mock_session.add.assert_called_once()
  - mock_session.commit.assert_called_once()
  - mock_session.refresh.assert_called_once()
  - *These confirm that the function correctly interacted with the database: it added the object, saved the changes (committed), and refreshed the object state.*
- **State Assertions**:
  - assert isinstance(result, Car): Ensures the function returns the correct object type.
  - assert result.doors == 2: Validates that the data passed in (doors=2) was correctly assigned to the resulting object.

What to Validate

- add() was called
- commit() was called
- refresh() was called

Also Validate

- Returned object type
- Correct data assignment

Why This is Important

- Tests logic in isolation
- Faster execution
- No external dependencies

Deployment Overview

There are many ways to deploy FastAPI applications. A common approach is using:

- **Linux server (e.g., Debian/Ubuntu)**
- **Nginx as reverse proxy**

Deployment Prerequisites

Before deployment, install:

- Python
- Git
- Nginx (light version is sufficient)

Deployment Architecture

Typical Setup

Client → Nginx → FastAPI (Uvicorn)

Role of Nginx

- Acts as reverse proxy
- Handles incoming traffic

- Improves performance
- Manages HTTPS

Docker-Based Deployment

Why Use Docker?

- Ensures consistent environment
- Simplifies deployment
- Encapsulates dependencies

What Docker Does

- Packages application + dependencies
- Runs as a container

Cloud Deployment

Modern deployments often use cloud providers.

Benefits

- Built-in HTTPS support
- Scalability
- Managed infrastructure

Examples

- Virtual machines
- Container orchestration platforms

End-to-End Deployment Flow

1. Write and test code
2. Package application (Docker)

3. Deploy to server/cloud
4. Configure Nginx
5. Enable HTTPS
6. Monitor logs